# Whole-Route Planning of a Memory Bus Between Two BGAs: An Exact Model of Crossings and Layer Changes, Negotiated to DRC-Clean Copper

Andrew Haas

Abstract—Routing the bus between a processor and its DRAM remains largely manual in printed circuit board (PCB) design, and published algorithms optimize proxies such as crossing or layer counts at the planning stage. We present a router that plans every lane of a two-layer BGA-to-BGA bus completely before any copper is laid, and carries the plan to finished, design-rule-clean copper. Its core is an exact constraint-programming model that chooses, for all lanes at once, where each necessary crossing occurs, the order of the crossings (consistent with a wiring diagram of the lanes' permutation by a rule over every triple of lanes), and where each lane changes layer, with via room and spacing; for the ends it is given, it proves the fewest layer changes within the model. The model reads surprisingly naturally as a problem in physics—worldlines, frustrated Ising spins and contact forces—and the connections proved of practical use. A linear program then places the lanes in millimeters; the constraints it must violate return to the model as cuts, and audit findings as negotiated congestion prices, until the plan passes an audit against the production router's rules or lays clean. The escapes at both arrays are chosen by an estimate of the same objective, and differential pairs are planned with their polarity at each end, with an automatic via crossover where the ends disagree. On two public boards, an Allwinner H3 and a Zynq-7020 each to a DDR3 device, the router completes every rung of a ladder of up to 51 nets—the H3's whole bus, and 43 of the Zynq's 48—connected and DRC-clean, never with more vias than the human designer (80 against 88 on the H3, 62 against 100 on the Zynq) and always with less copper. Run on the same benches, the toolkit's production router leaves nets open on four of the ten rungs, with up to 4.7 times the vias, and Freerouting completes none. On generated buses of three, four and six routing layers every run completes, and every one with a known optimum attains it.

Index Terms—Printed circuit board, bus routing, escape routing, topological via minimization, layer assignment, constraint programming, linear programming, differential pairs, DDR.

## I. Introduction

A memory bus is the densest wiring on most processor boards: tens of nets leave a ball grid array (BGA) in a fixed order, arrive at another in a different order, and must cross one another on the way. On a two-layer region every crossing forces one of the two nets onto the other layer there, and every layer change is a via—a drill, an annular ring that blocks both layers, and a discontinuity in the signal path. Commercial tools approach it interactively. In Cadence's Allegro, with its auto-interactive breakout, and in the sketch router of Mentor's (now Siemens') Xpedition, the designer sequences the bus as a bundle and plans its breakouts at both arrays together, resolving crossings by changing that sequence, a breakout's exit side or the pin assignment, and the tool fills in the route [2], [3].

The literature treats the problem in stages—escape routing, simultaneous escape routing, bus planning and layer assignment (Section II)—each optimizing a proxy such as crossings, layers or a planned via count, evaluated against other planners on industrial cases that cannot be reproduced. Whether a plan can be laid in copper at the board's clearance, and how its via count compares with a designer's, are left open.

This paper starts from an observation that designers, commercial tools and simultaneous escape routing share [2], [4]: the ends decide the braid. Once each net's escape at the source and destination is fixed, the orders of those escapes around the two arrays fix which pairs of nets must cross. What remains free is where each crossing happens, in what order the crossings occur, and where each lane changes layer. These choices interact—a via placed to serve one crossing serves or spoils the next, and two vias need room from each other and from every lane passing by—so we decide all of them at once, for every lane, before anything is routed. The contributions are:

1) An exact constraint-programming (CP) model of the whole route (Section IV) that chooses crossing positions, their order, and every lane's layer changes jointly. Its consistency rule over triples of lanes admits exactly the orders that a wiring diagram of the lanes' permutation can realize. For the ends it is given, the model proves the fewest layer changes within its scope.
2) A realization of the plan in board geometry (Section VI): an elastic linear program whose violated constraints return to the CP model as cuts, a snap onto the production router's grid, and an audit against the router's own rules, closed in a negotiation loop that also feeds findings at the ends back to the choice of escapes (Section V).
3) Differential pairs planned as single lanes with a polarity (handedness) at each end, and an automatic via crossover that swaps P and N where the two ends

A. Haas is a Professor in the Department of Physics, New York University, New York, NY 10003 USA (e-mail: andy.haas@nyu.edu).

The software, the benches and the recipes that reproduce every number in this paper are open source [1]; the results are those of its commit 09a3bf3b.

disagree.

4) An evaluation on two public boards against the human layouts, the toolkit's production router and Freerouting, every board graded on finished copper by the same independent design-rule and connectivity checks, and on generated buses of two to six routing layers against their exact optimum (Section VII).

The models have a surprisingly natural reading in physics. Read along the route, each lane is the worldline of a particle in one space and one time dimension, and each crossing a scattering event; the triple rule that keeps the crossings consistent is the Yang–Baxter relation of three strands. Each worldline carries an Ising spin, its layer, which must disagree with its partner's at every crossing: a via is a domain wall, and frustration is what forces vias; on more layers the spin becomes a Potts spin. Once the discrete choices are made, placing the lanes is a problem in statics—strings under tension, held apart by struts and walls—whose contact forces are the linear program's dual variables.

## II. Related Work

Topological via minimization. Given the nets' topology, choosing layers to minimize vias is classical. Hsu [5] decoupled topology from geometry to minimize vias; Marek-Sadowska [6] showed the unconstrained problem NP-complete, and Sarrafzadeh and Lee [7] gave a polynomial algorithm for nets between two shores, where the problem reduces to covering a permutation graph. Constrained via minimization assigns layers to an already routed layout [8]: it is NP-complete in general [9], and where the layout's junctions have degree at most three it is a maximum cut on a planar graph, solvable in polynomial time [8], [10], which Barahona et al. solved with the same code as Ising spin-glass ground states [11]. Exact algorithms followed for $k$ layers in channels without local nets [12] and, through maximum $k$-cofamilies of a partial order, in crossing channels [13]; in the redistribution layers of a package, routed without vias, Lin, Chi and Chang choose each connection's layer from the increasing subsequences of the routing order, minimizing the number of layers [14]. None of these places the crossings, orders them, or gives vias their room, jointly with the layers.

PCB escape and bus routing. Ozdal and Wong chose escape patterns at both ends of a channel jointly, with a longest-path search for each layer's planar subset and negotiation for the rest [4], and Ozdal, Wong and Honsinger minimized the vias the choice causes directly [15]. Bus planning and layer assignment for dense boards followed [16], [17]; where a bus's pin orders at its two ends disagree, Yan and Wong untangle the twisted nets with single detours near the ends, on one layer and with no vias [18], [19]. Exact satisfiability models route the escapes at one array [20], [21], a mixed-integer program routes two-layer BGA packages to their design rules over a restricted route structure [22], and differential pairs have escape routers of their own [23]. Most recently, Wu et al. [24] unified the escape and area regions in one circle model, searched escape orders by simulated annealing, and split unassigned nets with vias, reporting 40.9% fewer planned vias than a two-phase baseline. Lin et al. carried simultaneous escape routing through gridless area routing to finish commercial boards that a commercial router could not [25]. Apart from that flow and the escape routers, this work stops at a plan, and all of it is evaluated on private cases.

Topology, then geometry. Rubber-band routers [26] and the theory of homotopic routing [27] separate a topological route from its embedding. In the redistribution layers of a package, Wen et al. assign layers by a weighted maximum planar subset of chords, route the remaining nets through vias, and finish with a layout optimization by linear programming [28]. Negotiated congestion [29] prices places that were overused before. Our loop resembles logic-based and combinatorial Benders decomposition [30], [31], but its cuts are heuristic generalizations of a geometric failure, not proofs (Section VI-E).

Wiring diagrams. The orders in which the crossings of a permutation's reduced words can occur are the convex orders of its inversion set [32]; for the longest permutation their commutation classes form the higher Bruhat order $B(n, 2)$ [33]. We are not aware of a model that uses this structure to minimize vias jointly with crossing positions and via room.

## III. The Problem and the Frame

### A. Setting

A bench is a board region holding a source BGA $S$ (the processor) and a destination BGA $D$ (the DRAM), a set of bus nets between them, and everything else—passives, other nets' stubs, the board edge—as fixed obstacles. The region has two signal layers, F and B (Section VII-E extends the model to more). Each net leaves $S$ at a tooth, the end of a fanout stub from its ball, and arrives at $D$ at a berth; each end has a layer. The fanout engine offers every ball a menu of escape moves (a stub on one layer, or a dog-bone via to the other), from which the ends are chosen (Section V). A differential pair is one lane with two legs. A lane's layer change is a dive: one via for a single lane, two barrels side by side for a pair. The boards' clearance is 0.1 mm; we plan and route 5 $\mu$m above it, at 0.105 mm, with 0.127 mm lanes, 0.1 mm fanout stubs and 0.25 mm vias (0.15 mm drill) on a 0.025 mm grid. Two neighboring lanes on one layer must lie at least a bar of 0.232 mm apart, center to center (a lane and the clearance).

The objective counts planned vias: every layer change of a lane, and five more for every via a net carries beyond two (its stubs' vias included, a pair counted by its worse leg), so that a net of 0, 1, 2, 3 or 4 vias costs 0, 1, 2, 8 or 14; below a via, a congestion tie-break. These counts are of the plan, not the board: the stub vias come from the choice of ends (Section V), and a pair's layer change counts once although it lays two barrels. The result must be routed by the production router, connected, and clean under a design-rule check (DRC) at the board's clearance.

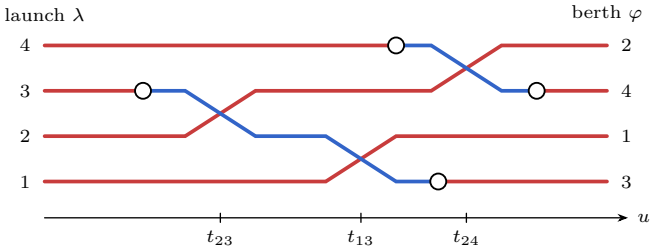



Fig. 1. Four lanes as a wiring diagram along the route coordinate $u$. The berth order is $\pi = (3, 1, 4, 2)$, so the inverted pairs $\{2,3\}, \{1,3\}, \{2,4\}$ must each cross once. On the triple $(1, 2, 3)$ only $\{1,3\}$ and $\{2,3\}$ are inverted, so the triple rule forces $t_{23} < t_{13}$. Red is F, blue is B; circles are vias. Crossing lanes are on different layers. Lane 3 serves its two F partners in one excursion to B, and lane 4 serves one: four vias, equal to the floor $2(K - \mathrm{LIS}(\pi)) = 2(4 - 2)$ of Lemma 1.

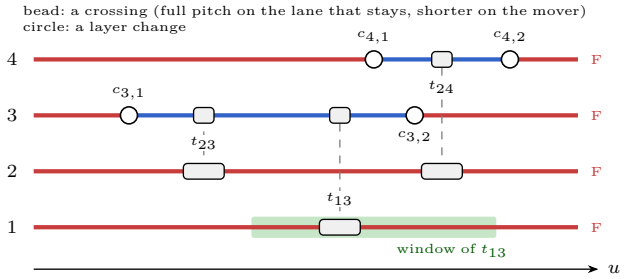



Fig. 2. The whole-route model as beads on tracks, for the four lanes of Fig. 1 (all ends on F). Each lane's route coordinate $u$ is a track, drawn in launch order. A crossing variable $t_{ab}$ is one bead that sits on both lanes' tracks at once (dashed tie), and intervals on one track may not overlap; each crossing slides only within its window (green, shown for $t_{13}$). Layer changes $c_{n,k}$ are beads too (circles), and between them a track's color is its layer. The model requires tied tracks to differ in color at their bead and each lane's number of changes to have the parity of its end layers, and the triple rule orders $t_{23}$ before $t_{13}$. The solver slides every bead at once and proves the fewest changes.

## B. The frame and the route coordinate

The frame is read off the board alone. A straight trunk runs from the pad box of $S$ through that of $D$; a ring follows the octilinear hull of $D$'s pads and stubs around each of the two faces of $D$ beside the trunk. Unrolling the perimeter of $S$ from a cut on its far face—at the gap whose order crosses the berths least—gives the launch order $\lambda$ of the teeth; unrolling that of $D$ from a cut at the middle of the widest gap on its far face gives the berth order $\varphi$. Where the ends model chose the cuts, the frame takes its cuts instead, so that model and solve share one order. Every lane's route is then one scalar coordinate $u$, arclength along a spine shared by all lanes: the trunk's straight spine, continued past each ring's handoff by arclength along that ring's own spine. Lanes at the same $u$ stand across the bus from one another, so a crossing's position is one number for both lanes. We discretize $u$ on a grid $G$ of a fifth of the least planned lane pitch $p = 0.252\,\mathrm{mm}$, about $0.05\,\mathrm{mm}$.

A pair of lanes $(a, b)$ with $\lambda(a) < \lambda(b)$ is inverted if $\varphi(a) > \varphi(b)$. Every inverted pair must cross; in our model it crosses exactly once, and no other pair crosses. The set $I$ of inverted pairs is the inversion set of the permutation $\pi$ that takes launch order to berth order (Fig. 1). We call the crossings, with the order in which they occur, the braid; unlike a braid in topology it records no over and under, which is what the layers decide.

## C. Two bounds

Lemma 1 (Floor): If both ends of every lane are on F, any routing within the frame—lanes that pass round neither array, so that they leave the source in the order $\lambda$ and reach the destination in the order $\varphi$—needs at least $2(K - \mathrm{LIS}(\pi))$ vias, where LIS is the length of the longest increasing subsequence.

Proof: Lanes that never change layer stay on F and so cannot cross one another; they form an increasing subsequence of $\pi$. Every other lane leaves F and returns, at least two vias. ∎

The floor is the classical maximum planar subset of the two-shore case [7], and it is weak: lanes that leave F also cross one another, and an excursion to B cannot serve a crossing with a partner that is itself on B. For a uniformly random permutation the LIS grows as $2\sqrt{K}$, with Tracy–Widom fluctuations of order $K^{1/6}$ [34]—the statistics of the KPZ class of growth processes [35]. Real buses are structured, but the scaling is a yardstick for how the floor grows. The next lemma counts a lane's vias exactly, given its partners.

Lemma 2 (Blocks): Fix the layers of lane $n$'s crossing partners at their crossings, in the order the crossings occur along $n$, and let both ends of $n$ be on F. The fewest layer changes of $n$ is twice the number of maximal runs of consecutive partners that are on F.

Proof: At each crossing $n$ must be on the partner's opposite layer: on B across a maximal run of F partners, on F across a run of B partners and at both ends. Each F run therefore needs one change into B and one out, and no change is needed elsewhere. ∎

The difficulty is therefore not placing a lane's vias given its partners, but choosing every lane's layers and crossing order together—which is what the whole-route model does.

# IV. The Whole-Route Solve

One CP-SAT [36] model decides every crossing and every layer change for all lanes at once (Fig. 2).

Crossings. For each $(a, b) \in I$ an integer $t_{ab} \in [\ell_{ab}, h_{ab}]$ on the grid $G$ is the crossing's position along the route. Its window starts after both lanes have entered the trunk, keeps a margin from both terminals, excludes the band in front of the source face where the teeth stand, and ends at the handoff to the rings unless both lanes share a ring. The triple rule: for every $i < j < k$ in launch order,

- if $ij, ik, jk \in I$: $t_{ij} < t_{ik} < t_{jk}$ or $t_{jk} < t_{ik} < t_{ij}$ (one Boolean);
- if $ij, ik \in I$ and $jk \notin I$: $t_{ij} < t_{ik}$;
- if $ik, jk \in I$ and $ij \notin I$: $t_{jk} < t_{ik}$.

The case $ij, jk \in I$, $ik \notin I$ cannot occur. The rule is a known characterization, stated here in the form the model uses:

Proposition 1: A linear order of $I$ is the order in which the crossings of some wiring diagram (reduced word) of $\pi$ occur if and only if it satisfies the triple rule.

Sketch: In a wiring diagram two lanes cross only when adjacent, so after any prefix of crossings the lanes stand in the order of some permutation, and the prefix is its inversion set; conversely, if a prefix and its extension by one pair are both inversion sets, that pair is adjacent and can cross next. A set of pairs is an inversion set if and only if, on every triple, it is closed ($ij, jk$ in the set imply $ik$) and co-closed ($ik$ implies $ij$ or $jk$); requiring this of every prefix, triple by triple, gives exactly the three cases above, which is Papi's characterization of convex orders [32]. ■

These are the convex (reflection) orders of the inversion set [32]; for the longest permutation their commutation classes form the higher Bruhat order $B(n, 2)$ [33]. Crossings of disjoint pairs may coincide—they commute—while crossings that share a lane are kept apart by a per-lane no-overlap constraint on intervals of the lane pitch. A Boolean per crossing chooses which lane stays and which moves across: the stayer's interval is the full pitch, the mover's the shorter footprint of its sweep. The two crossings of a pair lane that pass other lanes in opposite directions are kept a turning run apart.

Worldlines. Read $u$ as time and the lane order as position on a line: each lane is then the worldline of a particle in one space and one time dimension, and a crossing is a two-body scattering event. Three mutually crossing lanes can scatter in only two orders, $t_{ij} < t_{ik} < t_{jk}$ or its reverse—the two sides of the Yang–Baxter relation [37], or the third Reidemeister move. The triple rule's Boolean is the choice between them.

Layer changes. Each lane $n$ has up to three change positions $c_{n,1..3}$ with activity literals $a_{n,k}$; active changes are ordered and a via's room $D_v$ apart, and their parity is fixed by the ends:

$$\bigoplus_k a_{n,k} \;=\; \ell_n^S \oplus \ell_n^D, \tag{1}$$

where $\ell_n^S, \ell_n^D \in \{0, 1\}$ are the end layers. For each crossing $(a, b)$ and each change $k$ of lane $a$, a literal $\beta_{a,k,ab}$ states that the change lies before the crossing, enforced with clearance: $c_{a,k} \le t_{ab} - h$ or $c_{a,k} \ge t_{ab} + h$, where $h$ is the room of both lanes there (wider when the crossing lane is a pair, and dependent on the stay/move choice). The layer of $a$ at the crossing is $\ell_a^S \oplus \bigoplus_k (a_{a,k} \wedge \beta_{a,k,ab})$, and crossing lanes must differ:

$$\ell_a^S \oplus \ell_b^S \oplus \bigoplus_k (a_{a,k} \wedge \beta_{a,k,ab}) \oplus \bigoplus_k (a_{b,k} \wedge \beta_{b,k,ab}) \;=\; 1. \tag{2}$$

Changes of lanes that can be neighbors—adjacent in $\lambda$ or $\varphi$, or crossing each other—are staggered along $u$ by $\max(d, \sqrt{v_v^2 - p^2})$, where $d$ is the via's diameter and $v_v$ the least distance between two vias' centers (with a grid step's margin), so that two vias one pitch $p$ apart across the bus still meet it. A single lane may instead change layer right at the end of its stub, with no end room or stagger, but only where a via there is measured clear of every other net's copper on both layers: the fanout ends several stubs at one point of a channel's exit, and a via there would stand on the others.

Spins. Give each worldline an Ising spin, its layer. Equation (2) is then a hard antiferromagnetic bond at every crossing, a via is a domain wall along a worldline, and the objective counts domain walls. A cycle of must-differ constraints with an odd number of links cannot be satisfied without a wall—it is frustrated [38]—and frustration is what forces vias. With the crossing order fixed this is the planar spin-glass ground state that Barahona et al. solved as a maximum cut alongside via minimization [11], and Lemma 2 counts the domain walls of one worldline in the field of its partners. The whole-route solve also chooses the order of the scattering events.

Built-in cuts keep a lane's changes off stretches within a via's reach of another part's pad or a foreign tooth, and a pair's off its known corners and its end connectors (Section VI-C).

Objective. With $o_n \ge 0$ and $o_n \ge s_n + \sum_k a_{n,k} - 2$ counting the vias of lane $n$ beyond two ($s_n$ its stub vias, the larger leg's for a pair, less a tie via that joins a ball to a pad of its own net under it),

$$\min\; W_O \sum_n o_n \;+\; W_V \sum_{n,k} a_{n,k} \;+\; H, \tag{3}$$

with $W_V = 10^6$, $W_O = 5W_V$, and $H$ the history term of Section VI-E, a small fraction of a via for each place it prices. Preferring at most two vias per net is a price, not a cap: a cap is infeasible on many instances, and a lexicographic preference would buy one net's third via back with any number of other vias.

Proving it. CP-SAT's bound-raising workers—two linear-relaxation searches, a core-based search and an objective lower-bound search—run first, as a probe of ten interleaved batches, and their plan meeting their bound is the solve; otherwise the plan-finding workers continue from the probe's plan and bound, one search of at most a hundred batches that stops once the vias are proved. The four workers of CP-SAT's default portfolio include none that raises the bound, and a plan's vias are proved from below. A re-solve in the loop only adds cuts and history below a via, so the first solve's optimum is a valid floor for it, and a plan at that floor is proved at once. Every budget is a count of deterministic work, never a clock. A plan the search cannot prove is still kept and laid, its nets over two vias returned to the ends; on both ladders every first solve was proved (Section VII), and no plan changes a lane's layer more than twice.

Scope. The proof is relative to the model: the ends are fixed, each inverted pair crosses once (non-reduced braids are excluded), a lane changes layer at most three times (no plan on either ladder uses more than two), positions lie on $G$, and the loop's cuts are heuristic. What is proved is the fewest layer changes for the given ends. The board's via count also depends on the ends, whose stub vias are

about half of it, and a better choice of ends can beat a proved plan.

## V. Choosing the Ends

Because the ends decide the braid, the escapes at both arrays are chosen with an estimate of the solve's objective, to which the search adds prices of its own: the lanes' length at a fixed rate per via, trunk congestion and crossings, and the audits' feedback. For a candidate assignment of teeth and berths we estimate the route's vias from the ends alone: among lanes whose ends share a layer, inverted pairs on one layer must be separated by lanes that dive, and since their conflict graph is a permutation graph the cheapest cover is the complement of a maximum-weight increasing chain, exact per layer; a lane whose ends differ changes once, and the partners it would meet on the wrong layer form a bipartite graph covered exactly by a minimum cut; conflicts between such lanes are resolved greedily. Congestion adds each lane's load on the trunk (the room its crossings and changes take), priced by the square of its excess over half, and a fifth of a via per trunk crossing. The crossing price exists because the solve's effort follows its crossings and their triples, not its vias: on the H3 ladder the first solve proves 7 crossings at once, 126 in 5 s and 313 in 28 s (Section VII-F). The search moves one tooth or berth at a time (ejecting one conflicting lane where needed), then iterates from seeded random kicks. The best few states are then re-ranked exactly on their orders, by a lighter form of the solve's model: no crossing has a position; one Boolean orders the three crossings of each triple of mutually crossing lanes, each lane's crossings keep one transitive order, each layer change falls before or after each of its lane's crossings, and a run's layer is a Boolean. Lanes that never change layer and cross one another must lie on different layers, so of $c$ mutually crossing lanes on two layers at least $c - 2$ change layer; this bound, for every maximal clique of the crossing graph, is added to the model so that the search need not prove it, and CP-SAT searches core-guided, since nearly every lane has no change. When this form replaced one with a position per crossing, the sixteen states of the 51-net rung's ranking went from 192 s to 6 s. Each exact ranking stops at a work limit and the search is local, so the chosen ends are not proved best.

Street berths. A ball beside the unpopulated band of a DDR3 package may dive in a lane of the band and leave along it on the other layer, as the human does with 20 of its 30 signal vias at the DRAM. Offering such berths trades a few vias for fewer crossings, and so for a quicker solve.

## VI. From Plan to Copper

### A. Geometry

A single linear program places every lane: an offset per lane per column, columns every four grid steps (0.1 mm) along the trunk and both rings, joined where each lane's trunk ends. Constraints keep each column's lanes in the solve's order on their solved layers, same-layer neighbors a bar apart (slope-corrected by tangent cuts), a via's room around every change (chords of a disc at neighboring columns), every lane inside the board and off the pad boxes, and turns within 45° per column (per turning run for a pair). Costs are the added length—under-estimated by tangent cuts, so that a small sideways move costs next to nothing and lanes take a comfortable pitch where there is room—the bends, and a hinge penalty on neighbors closer than the comfortable pitch. Every hard constraint except the terminal pins, the end holds and the joins between trunk and rings is elastic, its slack priced at $W_H = 10^4$ per millimeter, so the program always has a solution and reports exactly where it had to break a rule.

A second pass holds each lane to one side of every island—a part's pads, or several parts no lane can pass between. The choice of side is non-convex, so it is made combinatorially before the pass, by neither the LP nor the CP-SAT model: at each island the lanes, in the order the CP-SAT solve fixed there, are split at the index that the first pass's offsets make cheapest, and the island is inserted into the order at that split. The sides are thus chosen consistent with the crossing plan, and the program stays convex; the polish may still flip a lane it cannot hold on its side (Section VI-B). Most of the second pass's rows are tangent cuts—of a column's length, a pitch, a via's room, each at several slopes—and few of them bind, so they are added lazily: the pass is solved with those near binding at the first pass's answer, then again with any its answer breaks, each solve warm-started from the last one's basis, until none is broken. The result is the whole program's optimum. The constraints paid in the second pass become cuts for the solve: a lane that could not be kept off an island may not cross anyone over the island's span; a change that could not be given its room may not lie there.

The program is large: pass one of the 51-net rung has 7 228 lane offsets, a slack or cost variable for each elastic constraint, and 150 753 constraints; pass two has 246 648, of which the lazy solves use 148 257. We pass its dual to HiGHS [39] through SciPy and read the offsets back as the multipliers of the dual's balance equations, falling back to the dual simplex when the interior point does not terminate. On the 51-net rung the two passes take 25 and 109 s, the second in four lazy solves.

Forces. The program's optimality conditions are those of a mechanism at rest (Fig. 3). Each lane point is a bead that slides on its column's rail, the length term is a string at unit tension, and the rules are contacts: struts between neighbors, walls at the pads and the board edge. A rule's dual value is the force in its contact, and complementary slackness is the contact law—a strut pushes only while it touches. An elastic rule is a rigid-plastic strut that carries up to $W_H$ and then yields. The static and kinematic theorems of plastic collapse are this duality [40], and the linear program that tests a packing of hard spheres for jamming solves for the same contact forces [41]. The

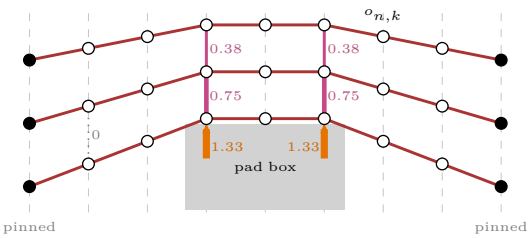



Fig. 3. A small instance of the geometry LP, solved. Three lanes on one layer cross nine columns, pinned at both ends (black), and must pass over a pad box. Each lane point slides on its column's rail (dashed); the lanes are strings at unit tension. The rules that bind carry the dual values drawn: the box (orange) pushes the lowest lane up where the lanes bend, and pitch struts (magenta) pass the load up the stack, each carrying the load of the lanes above it (their tension and bending; the toy also pulls each lane weakly toward its straight reference). Every other rule carries no force, like the strut near the pins (dotted), which has room to spare. Forces are in units of the string tension.

picture is useful for reading a plan. Only a small fraction of the hard constraints carry a force; together with the pinned ends and the length, bend and comfort terms, they hold every lane where it is, and they name the rules that do. A hard constraint the program could not keep has its elastic slack paid—a strut that yields—and those of the static, via and dive kinds are what the geometry returns to the solve as cuts.

## B. Polish and snap

The polish meets the audit's own measures in board coordinates (pitch, via room, clearance to pads as drawn, turn shapes) by small vertex moves, one linear program per round within a trust region, with every bar between two planned lanes a grid step above the snap's, and half a step against static copper. Every round has an answer: a rule enters the program while it stands within twice the trust of its bar, so none is broken out of sight; a turn the round cannot open in one step is opened as far as the trust allows and the rest paid; a lane or via found inside a pad is pushed out of the pad's island, not toward its nearest edge; and the state kept is the best any round reached, not the last. A lane that cannot be held on its side of an island is flipped, and the geometry runs again.

The snap makes the smooth plan octilinear on the router's grid, one lane at a time, by an A* search in a strip around each lane's smooth line that prices length, bends and distance from the line. Pairs go first, moving exactly as the pair router moves (45° turns, a straight run after each turn, a via only on a straight run); they are then held while the polish fits the singles around them. A lane not yet laid keeps a share of every gap—the side of the midline nearer its own line, less half a bar—so lanes laid first cannot take the room later ones need. Two sweeps then lay every lane again against the others' real copper.

## C. Differential pairs

A pair is one lane of two slots' width. Its hand at an end is the side of its travel on which P lies. A pair whose hands at its tooth and berth disagree must swap its legs somewhere; the solve then requires it to change layer at least once, and the snap lays a crossover at its first dive, as a designer would: the first leg jogs out at 45° to its barrel and dives, the second jogs at 45° over the first leg's new-layer segment to a barrel just beyond and returns to its line. Both barrels lie on one side of the pair, staggered by the fewest whole grid steps that keep a via's pitch and each jog's clearance, and every stage knows the crossover's footprint before it is laid. At each end the legs converge, through an end connector—two legs from the pair's tips, turning at most 45°—onto a pose, a grid point and heading from which the pair router takes over, so plan and router share one end.

## D. Audit and route

The audit checks the plan against the router's rules as the router will see them (pads as drawn, each terminal joined as the router joins it), and the gate passes it only when it is complete and clean with nothing waived. The production router [1] then lays the pairs, then the singles, each lane confined to its band—the router cells within half a grid step of its snapped line, with its end and via cells; a pair the router refuses in its band is tried in a wider one, and free of the plan last, after every lane has had its band, so that it cannot take the room the plan gave the singles. The audit models the router's rules but is not the router: a plan that passes the gate can still have a lane refused, which is why the board, not the plan, is graded. Independent checks grade connectivity and DRC on the written board.

## E. The negotiation loop

Every stage is fed a measurement of the one before (Fig. 4). The audit's findings return to the solve as history: every place a round found the plan short is priced, as a negotiated router prices a place overused before [29], by how many audits found it so: about $10^{-4}$ of a via for a crossing there and $10^{-3}$ for a layer change, for each audit. History thus reorders plans of equal vias; its total is not capped, so on a bus with very many findings it could in principle outweigh a via, and the solve's proof, which reads the objective in whole vias, assumes it does not. The geometry's paid constraints and the snap's failures return as cuts; side flips return to the geometry. Cuts that together leave no plan are kept as soft costs of three vias each—above a dive's two, below a net's third via, and whole, since the proof reads the objective in whole vias. The loop stops when the gate passes, after two rounds that do not improve on the best, or when a finding stands at the ends, which only the fanout can move: the finding is then priced in the ends model by place, doubling when the same end is named again, and the next fanout round chooses again, from the previous round's board with only the named ends, and both ends of any lane over two vias, free.

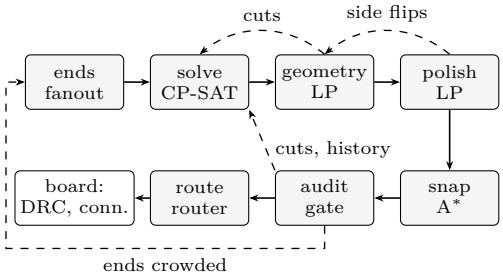



Fig. 4. The whole route. Solid arrows pass the plan forward; dashed arrows are the negotiation: the rules the geometry had to break return to the solve as cuts, the audit's findings between the arrays as cuts and history, lanes with no room beside an island return to the geometry as side flips, and findings at the ends return to the choice of escapes.

Never nothing. A loop that does not pass is still laid, from the best plan it held, and a lane that cannot be laid stays open; a round that lays connected and DRC-clean ends the run, whether or not its loop passed and its solve proved the plan, since a later round could change only ends the lay already went round. A round that leaves nets open sends them back to the fanout, and the run's result is its best round: the fewest open nets, then the fewest vias. This is a counterexample-guided loop in the spirit of combinatorial Benders decomposition [30], [31]; unlike it, a cut here generalizes one geometric failure and is not proved valid, so the loop's result is optimal only for the model it ends with.

Sectors. The route's non-convexity lives entirely in discrete choices—the crossing order and the side of each island, together a homotopy class of the whole bus. Within one such topological sector, and with the first pass's linearization fixed, the geometry is convex, and a small tie-break picks one point of its face of optima. The loop is a search over sectors: the solve proposes one, the geometry relaxes within it, and the constraints that yield tell the solve which sectors to leave.

## VII. Results

### A. Benches and grading

H3. An Allwinner H3 (0.65 mm ball pitch) to a DDR3 device on a public reference design [42], whose designer routed the bus on its two layers. Its bench carries 51 bus nets, among them three differential pairs (two data strobes and the clock); the ladder's rungs K15–K51 add whole groups of nets, tightest first. Zynq. The Zynq-7020 (CLG400) and its DDR3 on a public design [43], whose designer routed the bus on the outer layers only (the inner layers are planes), every via F to B. We strip its routing, fan out the processor and keep the two outer layers, by a recipe from the board file in the repository [1]; the bus has 48 nets, and the ladder's rungs K18–K43 carry both strobe pairs. (The full bus, K48, is left out: on it the whole route leaves two or three nets open in each of its three fanout rounds.)

Each result is written as a board and graded independently: connectivity of every bus net, DRC at the board's clearance of 0.1 mm, and a census of every via and millimeter of the run's nets, stubs included. The other routers' boards and the human's are graded the same way; the human's copper includes length-matching meanders, which add copper but no vias. Runs used CP-SAT from OR-Tools 9.15 and HiGHS through SciPy 1.16, on Python 3.14, on an Apple M3 laptop (8 cores, 8 GB), at the toolkit's commit 09a3bf3b.

### B. Against the human

Table I gives every rung of both ladders on our own ends. Every rung routes every lane in its band at once, every net connected, DRC-clean, never with more vias than the human and always with less copper; every rung but H3 K28 and K35, whose first fanout the fanout's own audit refused, does so in its first fanout round. At the top of the H3 ladder the route uses 80 vias to the human's 88 (Fig. 5); on the Zynq, 62 to 100 (Fig. 6). Part of the copper difference is the human's length matching.

### C. Against other routers

Two other routers ran on the same benches, nets and design rules, and their boards were graded the same way (Table I): the production chain of the toolkit the whole route belongs to [1], without the bus step—its BGA fanout at the destination, its differential-pair router and its negotiated A* router—and Freerouting 2.4.1 [44], an open-source autorouter, with its default settings (its own BGA fanout and optimizer included), through KiCad's Specctra export. Freerouting routes each leg of a differential pair as an independent net.

The production chain completes six of the ten rungs and leaves one to four nets open on the others, with 2.0 to 4.7 times the whole route's vias (319 against 80 at H3 K51) and more copper on every rung; at Zynq K43 its board also has seven clearance violations. Freerouting completes none: it leaves 1 to 13 nets open, and at Zynq K33 its board also necks four tracks below the board's minimum width. On the nets it does connect it can use fewer vias than the whole route—44 against 54 at H3 K41, with four nets open—because its lanes enter the destination's ball field from the near face, where our ends escape every ball to the array's edge (Section VIII). Of the 50 nets it leaves open over the two ladders, 33 are address, command, strobe and clock nets. The production chain takes 4 to 72 s a rung, and Freerouting 1.5 to 5.5 minutes.

### D. Synthetic buses

The repository's generator plants a crossing pattern between two arrays and computes the optimum of its channel—every tooth on the source's facing column and every berth on the destination's, in the balls' order—exactly, over every crossing order [1]. Twenty-nine such buses of 4 to 16 nets span sorted, blocked, interleaved,

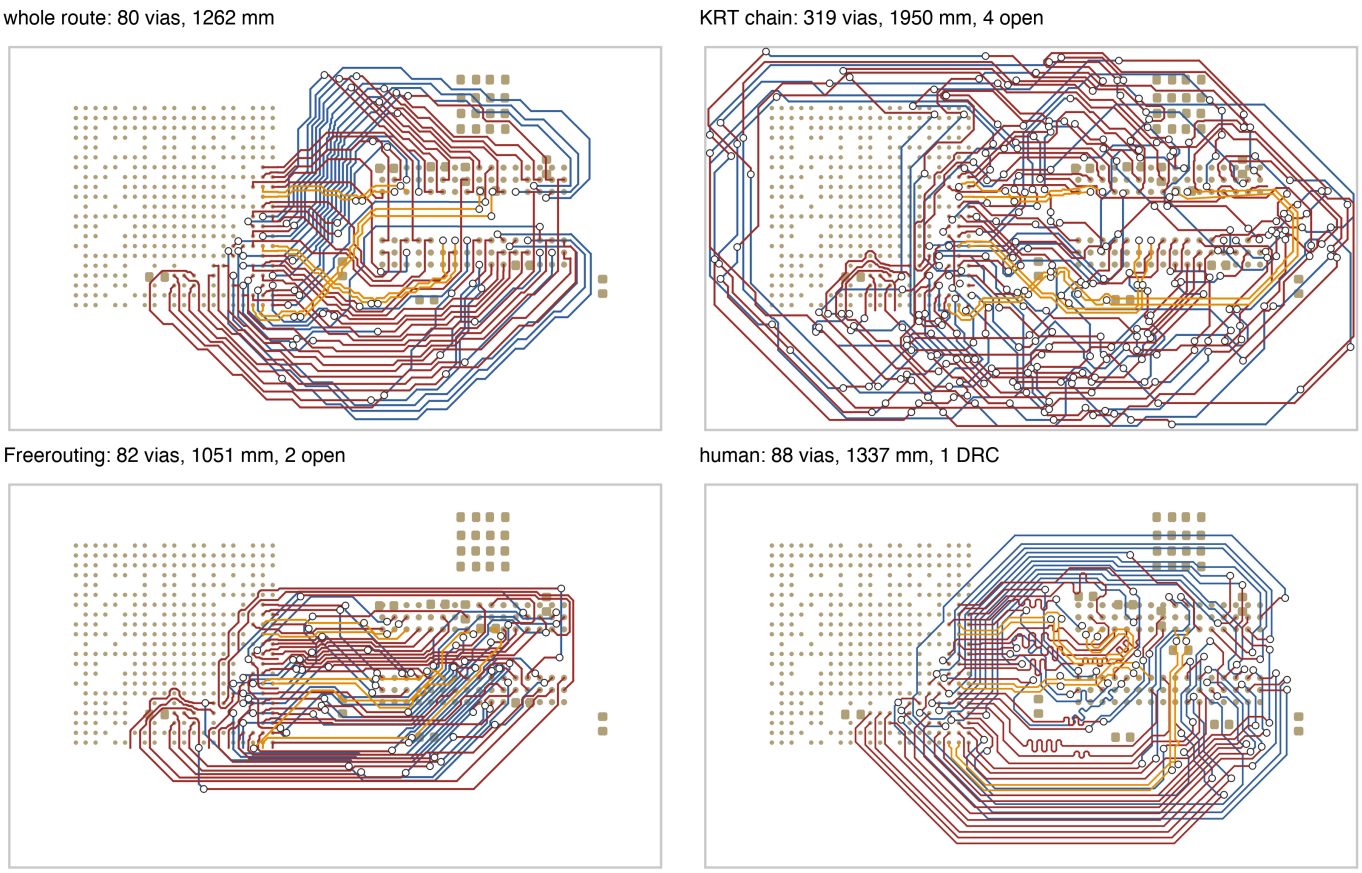



Fig. 5. The 51-net H3 rung by the whole route on its own ends, the toolkit's production router without the bus step, Freerouting, and the human, only the run's nets drawn; each panel is labeled with its vias, copper, open nets and DRC violations. F is red, B blue, the three differential pairs orange; every via is drawn as the same marker. The source BGA is at left, the DDR3 at right.

TABLE I
Vias, copper (mm), open nets and DRC violations of the run's nets on the finished board: the whole route, the toolkit's production router without the bus step (KRT chain), Freerouting 2.4.1, and the human's board, every board graded the same way.

| | whole route | | | | KRT chain | | | | Freerouting | | | | human | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| rung | vias | mm | open | DRC | vias | mm | open | DRC | vias | mm | open | DRC | vias | mm | open | DRC |
| Allwinner H3 to DDR3 | | | | | | | | | | | | | | | | |
| K15 | 12 | 194 | 0 | 0 | 40 | 424 | 0 | 0 | 16 | 153 | 1 | 0 | 22 | 232 | 0 | 0 |
| K28 | 32 | 561 | 0 | 0 | 72 | 877 | 0 | 0 | 29 | 416 | 5 | 0 | 48 | 678 | 0 | 1 |
| K35 | 44 | 758 | 0 | 0 | 138 | 1340 | 0 | 0 | 41 | 631 | 3 | 0 | 60 | 889 | 0 | 1 |
| K41 | 54 | 967 | 0 | 0 | 254 | 1749 | 2 | 0 | 44 | 752 | 4 | 0 | 70 | 1081 | 0 | 1 |
| K51 | 80 | 1262 | 0 | 0 | 319 | 1950 | 4 | 0 | 82 | 1051 | 2 | 0 | 88 | 1337 | 0 | 1 |
| Zynq-7020 to DDR3 | | | | | | | | | | | | | | | | |
| K18 | 18 | 480 | 0 | 0 | 36 | 562 | 0 | 0 | 20 | 307 | 6 | 0 | 45 | 653 | 0 | 0 |
| K26 | 28 | 761 | 0 | 0 | 72 | 800 | 0 | 0 | 24 | 427 | 10 | 0 | 57 | 915 | 0 | 0 |
| K33 | 44 | 990 | 0 | 0 | 104 | 1131 | 0 | 0 | 39 | 590 | 13 | 4 | 77 | 1197 | 0 | 0 |
| K38 | 56 | 1100 | 0 | 0 | 128 | 1290 | 1 | 0 | 58 | 1007 | 2 | 0 | 87 | 1371 | 0 | 0 |
| K43 | 62 | 1299 | 0 | 0 | 151 | 1601 | 1 | 7 | 64 | 1087 | 4 | 0 | 100 | 1577 | 0 | 0 |

riffled, shuffled and reversed patterns, short 3 mm channels, escapes that are all vias, open array interiors, walls of parts in the channel, rings round the destination, pairs, and lanes that must wind round it. On two layers the whole route completes 23 of them connected and DRC-clean. Of the 16 whose channel optimum applies, it lays exactly the optimum on 10 and completes two more above it, at 18 and 34 vias where 10 and 14 are optimal, with every escape a via. The six it leaves incomplete all have the short channel, and leave one to five nets open: two layers do not give the changes room there. On cases without a graded optimum the route can lay fewer vias than the channel's floor, because it chooses its own ends and may leave an array by another face.

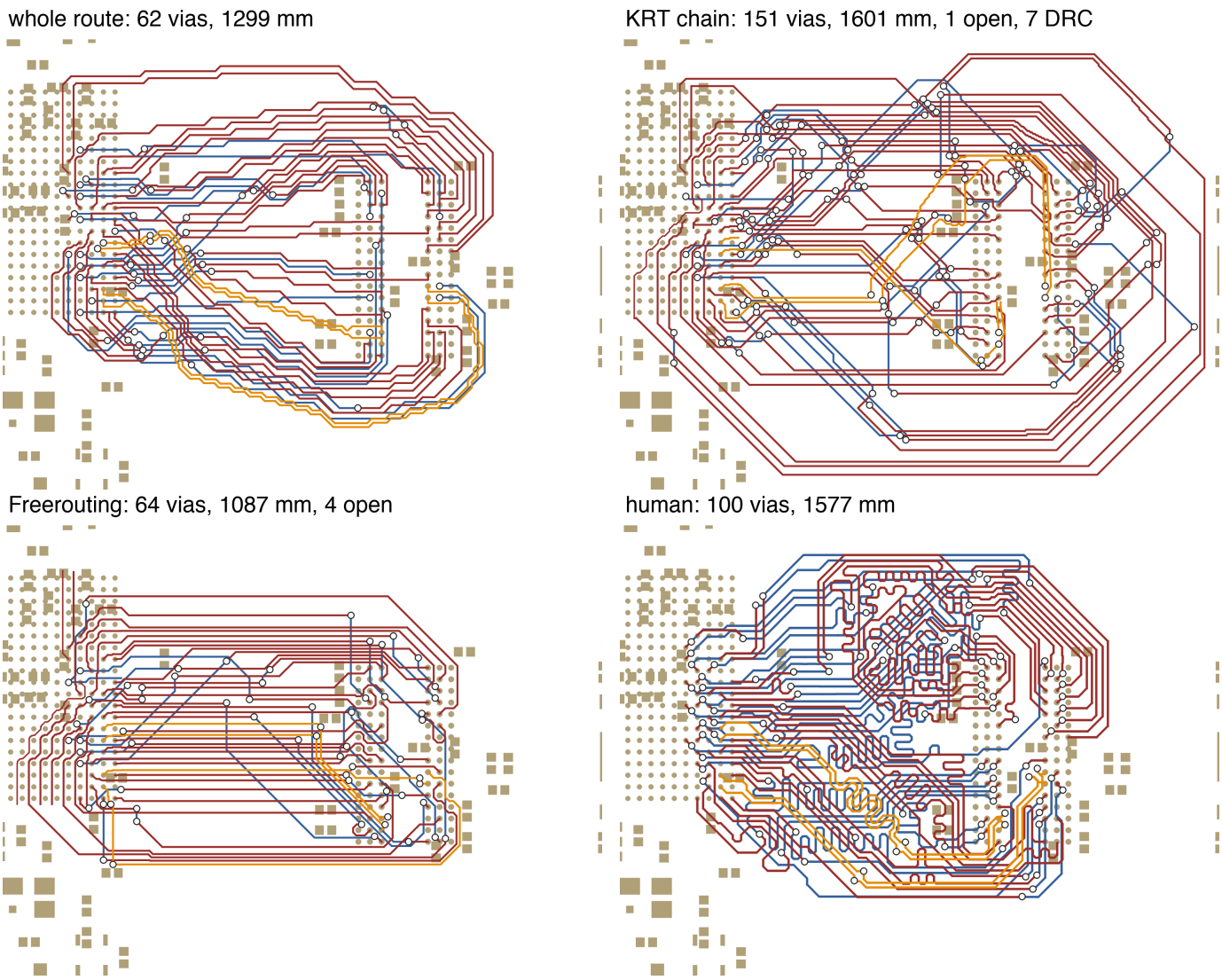



Fig. 6. The 43-net Zynq rung, drawn as Fig. 5 (its two strobe pairs orange), in the bench's frame (the human's board turned with it). Much of the human's copper is length-matching meanders.

### E. More routing layers

The model extends to $q > 2$ routing layers with few changes. A lane's layer between its changes takes one of $q$ values, two lanes cross only on different layers, and a crossing's room is shared only by the lanes that cross on one layer. An end whose stub carries a via may leave on any layer its run fits, the solve choosing which, and the run is moved onto that layer before the geometry. In the spin picture the layer becomes a $q$-state Potts spin [45] with an antiferromagnetic bond at every crossing, and frustration begins only at $q+1$ mutually crossing lanes: the clique bound of Section V becomes $c-q$ changes among $c$ such lanes. On the 29 generated buses on three, four and six routing layers, all 87 runs complete connected and DRC-clean, and all 48 whose channel optimum applies lay exactly that optimum, the short channels and all-via escapes included. More layers do not lower these channels' optimum, which equals the floor of Lemma 1 on every layer count since the arrays' pads are on F; what they give is room for the changes.

### F. Cost

Most of the time is spent choosing the ends: of K51's 721 s, the fanout with its exact re-ranking takes 488 s, the solve 28 s, and the loop's single round 192 s, of which the geometry LP takes 138 s. The cost grows with the rung: the 15-net H3 rung has 7 crossings and takes 61 s and 266 MB at its peak, the 51-net rung 313 crossings, 721 s and 741 MB, and the 43-net Zynq rung 172 crossings, 677 s and 913 MB, on the laptop running three rungs side by side, so that wall time includes waiting. Every rung's first solve is proved optimal: at the top of the H3 ladder its 313 crossings and 2 744 triples in 28 s, every other rung's in 6 s or less.

## VIII. Limitations and Future Work

The route does not match lengths; a DDR interface requires length (or delay) matching within byte lanes

and of the address group to the clock [46], and the human's copper includes that work. Matching lengths on the planned lanes, where the comfortable pitch leaves room for meanders, is our next step. Recent any-direction length matching assigns each trace a region of its own by linear programming and then meanders it within that region [47], [48]; we plan to make each lane's meander area a variable of the geometry LP, with the groups' length targets among its constraints, so that a target it cannot meet returns to the model as a cut. On more than two layers the model is so far graded only on generated buses (Section VII-E); on real boards with more layers it is being made a step of the toolkit's production chain, which routes everything else round the bus it lays. Optimality is relative to the model (fixed ends, reduced braids, at most three changes per lane, heuristic cuts), and the ends are chosen by a local search whose estimate is exact only in its parts. On boards whose escapes are all dog-bones the toolkit already chooses every lane's two ends together in one CP-SAT model, but over a proxy of the route's objective; a model over the ends that bounds the route's own could prove them. Our ends escape every ball to the array's edge; Freerouting's lanes enter the destination's ball field from the near face, which on the nets it completes costs fewer vias, so ends inside the ball field are a natural extension of the ends model. A DDR controller usually lets the data bits within a byte lane, and often whole byte lanes, be swapped; since the crossings come from the ends' permutation, letting the ends search choose that assignment—as pin assignment has been chosen jointly with escape routing [49]—could remove most of the crossings within a byte lane, and their vias. With the crossing order fixed and via room set aside, the layer assignment is a planar Ising ground state, a minimum T-join on the planar dual of the wiring diagram, and so solvable in polynomial time [10], [50], [51]; it could give the ends search a fast exact evaluator and a lower bound. The solve counts a pair's dive once, where the board counts two barrels, so its equal optima can differ on the board by two vias. Intra-pair skew is not yet planned. Finally, two boards are few, and the design constants were developed on them; we are extending the evaluation to more boards.

## IX. Conclusion

Planning a bus whole—ends, crossings, their order and every layer change, decided at once, with the fewest layer changes proved for the chosen ends, and negotiated with the geometry until the production router can lay it—routes one real DDR3 bus whole and 43 of another's 48 nets fully automatically, with fewer vias and less copper than their designers. The central model is small and exact because the problem's combinatorics are those of a wiring diagram, whose consistency is a rule over triples. Throughout this work the synergies between routing and physics proved abundant, interesting and of practical use: the worldline picture gives the triple rule its meaning, the spin picture traces every via to frustration, and the forces of the linear program's dual name the rules that hold each lane and the ones that must yield. We expect the same pairing—an exact choice of topological sector, relaxed in a convex statics problem whose forces steer the next choice—to carry over to length matching, to boards with more layers and to the other dense interconnects of modern hardware, and the dialogue between routing and physics to keep paying in both directions.

## References


[1] A. Haas, "KiCadRoutingTools," [Online]. Available: https://github.com/drandyhaas/KiCadRoutingTools, 2026, commit 09a3bf3b.

[2] R. S. Lawson, B. A. Neal, D. K. Morgan, and J. Radumilo-Franklin, "System and method for electronic design routing between terminals," U.S. Patent 9 202 001, Dec. 1, 2015.

[3] C. Pfeil, "What's the difference: PCB routing now and then," Electronic Design, [Online]. Available: https://www.electronicdesign.com/resources/whats-the-difference-between/article/21800231/whats-the-difference-pcb-routing-then-and-now, Sep. 2014.

[4] M. M. Ozdal and M. D. F. Wong, "Algorithms for simultaneous escape routing and layer assignment of dense PCBs," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 25, no. 8, pp. 1510–1522, 2006.

[5] C.-P. Hsu, "Minimum-via topological routing," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 2, no. 4, pp. 235–246, 1983.

[6] M. Marek-Sadowska, "An unconstrained topological via minimization problem for two-layer routing," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 3, no. 3, pp. 184–190, 1984.

[7] M. Sarrafzadeh and D. T. Lee, "A new approach to topological via minimization," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 8, no. 8, pp. 890–900, 1989.

[8] R. Y. Pinter, "Optimal layer assignment for interconnect," J. VLSI Comput. Syst., vol. 1, no. 2, pp. 123–137, 1984.

[9] N. J. Naclerio, S. Masuda, and K. Nakajima, "The via minimization problem is NP-complete," IEEE Trans. Comput., vol. 38, no. 11, pp. 1604–1608, 1989.

[10] F. O. Hadlock, "Finding a maximum cut of a planar graph in polynomial time," SIAM J. Comput., vol. 4, no. 3, pp. 221–225, 1975.

[11] F. Barahona, M. Grötschel, M. Jünger, and G. Reinelt, "An application of combinatorial optimization to statistical physics and circuit layout design," Oper. Res., vol. 36, no. 3, pp. 493–513, 1988.

[12] C. S. Rim, T. Kashiwabara, and K. Nakajima, "Exact algorithms for multilayer topological via minimization," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 8, no. 11, pp. 1165–1173, 1989.

[13] J. Cong and C. L. Liu, "On the $k$-layer planar subset and topological via minimization problems," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 10, no. 8, pp. 972–981, 1991.

[14] T.-C. Lin, C.-C. Chi, and Y.-W. Chang, "Redistribution layer routing for wafer-level integrated fan-out package-on-packages," in Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD), 2017, pp. 561–568.

[15] M. M. Ozdal, M. D. F. Wong, and P. S. Honsinger, "Simultaneous escape-routing algorithms for via minimization of high-speed boards," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 27, no. 1, pp. 84–95, 2008.

[16] H. Kong, T. Yan, and M. D. F. Wong, "Automatic bus planner for dense PCBs," in Proc. 46th ACM/IEEE Design Automat. Conf. (DAC), 2009, pp. 326–331.

[17] J.-T. Yan, "Layer assignment of buses and nets with via-count constraint in high-speed PCB designs," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 38, no. 3, pp. 512–525, 2019.

[18] T. Yan and M. D. F. Wong, "Untangling twisted nets for bus routing," in Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD), 2007, pp. 396–400.

[19] ——, "Theories and algorithms on single-detour routing for untangling twisted bus," ACM Trans. Design Autom. Electron. Syst., vol. 14, no. 3, pp. 1–21, 2009.
[20] L. Luo and M. D. F. Wong, "Ordered escape routing based on Boolean satisfiability," in Proc. Asia South Pacific Design Automat. Conf. (ASP-DAC), 2008, pp. 244–249.
[21] S. Bayless, H. H. Hoos, and A. J. Hu, "Scalable, high-quality, SAT-based multi-layer escape routing," in Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD), 2016, pp. 1–8.
[22] Y. Tomioka, Y. Kurata, Y. Kohira, and A. Takahashi, "MILP-based efficient routing method with restricted route structure for 2-layer ball grid array packages," IEICE Trans. Fundamentals Electron., Commun. Comput. Sci., vol. E92-A, no. 12, pp. 2998–3006, 2009.
[23] T. Yan, P.-C. Wu, Q. Ma, and M. D. F. Wong, "On the escape routing of differential pairs," in Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD), 2010, pp. 614–620.
[24] H. Wu, X. Jiang, Y. Fan, X. He, and M. Liu, "Topology optimization-based layer assignment method for fixed-layer complex PCBs," IEEE Trans. Compon., Packag., Manuf. Technol., vol. 16, no. 1, pp. 20–31, 2026.
[25] S.-T. Lin, H.-H. Wang, C.-Y. Kuo, Y. Chen, and Y.-L. Li, "A complete PCB routing methodology with concurrent hierarchical routing," in Proc. 58th ACM/IEEE Design Automat. Conf. (DAC), 2021, pp. 1141–1146.
[26] W. W.-M. Dai, T. Dayan, and D. Staepelaere, "Topological routing in SURF: Generating a rubber-band sketch," in Proc. 28th ACM/IEEE Design Automat. Conf. (DAC), 1991, pp. 39–44.
[27] C. E. Leiserson and F. M. Maley, "Algorithms for routing and testing routability of planar VLSI layouts," in Proc. 17th ACM Symp. Theory Comput. (STOC), 1985, pp. 69–78.
[28] H.-T. Wen, Y.-J. Cai, Y. Hsu, and Y.-W. Chang, "Via-based redistribution layer routing for InFO packages with irregular pad structures," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 41, no. 12, pp. 5554–5567, 2022.
[29] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in Proc. ACM Int. Symp. Field-Programmable Gate Arrays (FPGA), 1995, pp. 111–117.
[30] J. N. Hooker and G. Ottosson, "Logic-based Benders decomposition," Math. Program., vol. 96, no. 1, pp. 33–60, 2003.
[31] G. Codato and M. Fischetti, "Combinatorial Benders' cuts for mixed-integer linear programming," Oper. Res., vol. 54, no. 4, pp. 756–766, 2006.
[32] P. Papi, "A characterization of a special ordering in a root system," Proc. Amer. Math. Soc., vol. 120, no. 3, pp. 661–665, 1994.
[33] Y. I. Manin and V. V. Schechtman, "Arrangements of hyperplanes, higher braid groups and higher Bruhat orders," in Algebraic Number Theory – in honor of K. Iwasawa, ser. Adv. Stud. Pure Math., 1989, vol. 17, pp. 289–308.
[34] J. Baik, P. Deift, and K. Johansson, "On the distribution of the length of the longest increasing subsequence of random permutations," J. Amer. Math. Soc., vol. 12, no. 4, pp. 1119–1178, 1999.
[35] M. Prähofer and H. Spohn, "Universal distributions for growth processes in 1+1 dimensions and random matrices," Phys. Rev. Lett., vol. 84, no. 21, pp. 4882–4885, 2000.
[36] L. Perron and F. Didier, "CP-SAT," Google OR-Tools, v9.15. [Online]. Available: https://developers.google.com/optimization/cp/cp_solver/, 2025.
[37] R. J. Baxter, "Partition function of the eight-vertex lattice model," Ann. Phys., vol. 70, no. 1, pp. 193–228, 1972.
[38] G. Toulouse, "Theory of the frustration effect in spin glasses: I," Commun. Phys., vol. 2, pp. 115–119, 1977.
[39] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," Math. Program. Comput., vol. 10, no. 1, pp. 119–142, 2018.
[40] A. Charnes, C. E. Lemke, and O. C. Zienkiewicz, "Virtual work, linear programming and plastic limit analysis," Proc. R. Soc. Lond. A, vol. 251, no. 1264, pp. 110–116, 1959.
[41] A. Donev, S. Torquato, F. H. Stillinger, and R. Connelly, "A linear programming algorithm to test for jamming in hard-sphere packings," J. Comput. Phys., vol. 197, no. 1, pp. 139–166, 2004.
[42] hvwyl, "allwinner-h3-ddr3-16x1-kicad: Allwinner H3 with DDR3 16x1 design reference for KiCad," [Online]. Available: https://github.com/hvwyl/allwinner-h3-ddr3-16x1-kicad, 2023.
[43] kangyuzhe666, "ZYNQ7010-7020_AD9363: Open-source SDR hardware based on Zynq and AD9363," [Online]. Available: https://github.com/kangyuzhe666/ZYNQ7010-7020_AD9363, 2020.
[44] Freerouting contributors, "Freerouting," [Online]. Available: https://github.com/freerouting/freerouting, 2026, v2.4.1.
[45] F. Y. Wu, "The Potts model," Rev. Mod. Phys., vol. 54, no. 1, pp. 235–268, 1982.
[46] M. M. Ozdal and M. D. F. Wong, "A length-matching routing algorithm for high-performance printed circuit boards," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 25, no. 12, pp. 2784–2794, 2006.
[47] J. Lin, L. Guo, and W. Fang, "LP-based area assignment for length-matching routing of complex multilayer PCBs with any-direction wires," ACM Trans. Design Autom. Electron. Syst., vol. 31, no. 5, pp. 1–22, 2026.
[48] W. Fang, L. Guo, J. Lin, S. Xiong, H. He, J. Xu, and J. Chen, "Obstacle-aware length-matching routing for any-direction traces in printed circuit board," in Proc. 61st ACM/IEEE Design Automat. Conf. (DAC), 2024, pp. 1–6.
[49] S.-I. Lei and W.-K. Mak, "Simultaneous constrained pin assignment and escape routing considering differential pairs for FPGA-PCB co-design," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 32, no. 12, pp. 1866–1878, 2013.
[50] F. Barahona, "On the computational complexity of Ising spin glass models," J. Phys. A: Math. Gen., vol. 15, no. 10, pp. 3241–3253, 1982.
[51] M. Glass and M. Feder, "Maximum cut algorithms and upper bounds for planar and toroidal graphs," arXiv:2606.28478, 2026.